ASSIGNMENT 1: TABULAR REINFORCEMENT LEARNING

Student name
Manasvi Kumar

Student number
S3938425

1 INTRODUCTION

Tabular reinforcement learning aims to learn by interaction in simpler, low dimensional, environments such as Grid worlds and aims to find the best policy to operate in the environment. It focuses on sequential decision problems that can be modelled as Markov decision processes. The agent is guided towards the optimal policy by using the value function.

2 DYNAMIC PROGRAMMING

Richard Bellman introduced the term Dynamic Programming, with the aim to recursively traverse the states and actions. It uses the principle of divide and conquer: The main problem whose value is to be determined by searching a large subtree is broken down into sub problems which are then optimized to find a final solution.
The $Bellman\ equation$ is used to find the relationship between the value function in state s and the future child state s'. The equation is expressed as follows:-

$V^{\pi}(s) = \sum_{a \in A} \pi(a|s) \left(\sum_{s' \in S} T_a(s, s') \left(R_a(s, s') + \gamma \cdot V^{\pi}(s') \right) \right)$
where,

- π is the probability of action a in state s
- T is the stochastic transition function that determines how often the selected state occurs, R is the reward function giving the reward for the action.
- γ is the discount rate balancing the immediate and future rewards. If set to 0, the agent only regards current rewards, if set to 1 then it will check for high rewards in long run.

2.1 PROCEDURE

- Initialize value function $V(s)$ values to zero.
- call the state values, action values and rewards function for the task.
- calculate the maximum possible reward that can be obtained by taking each action for a particular state.
- Loop over the states and their actions while repeatedly updating $Q(s, a)$ and $V(s)$ values.
- Do until convergence

Once convergence is achieved, we can obtain the optimal policy by selecting the action with the highest Q-value for each state. The complete algorithm that we used for this task is as follows:-

---

Algorithm 1: Tabular Q-value iteration (Dynamic Programming)

Input: Threshold $\eta \in R^+$
Result: The optimal value function $Q^*(s, a)$ and/or associated optimal policy $\pi^*(s)$
Initialization: A state-action value table $Q^{\wedge}(s, a) = 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
repeat

1. $\Delta \leftarrow 0$
2. **for each** $s \in \mathcal{S}$ **do**
3. **for each** $a \in \mathcal{A}$ **do**
4. $x \leftarrow Q(s, a)$
5. $Q^{\wedge}(s, a) \leftarrow \sum_{s'} p(s' \mid s, a) \left[r(s, a, s') + \gamma \max_{a'} Q^{(}s', a') \right]$
6. $\Delta \leftarrow \max(\Delta, |x - Q^{\wedge}(s, a)|)$
7. **end for**
8. **end for**

until $\Delta < \eta$
$Q^*(s, a) = Q^{\wedge}(s, a)$
$\pi^*(s) = \arg\max_a Q^*(s, a) \quad \forall s \in S$
return: $Q^*(s, a)$ and/or $\pi^*(s)$.

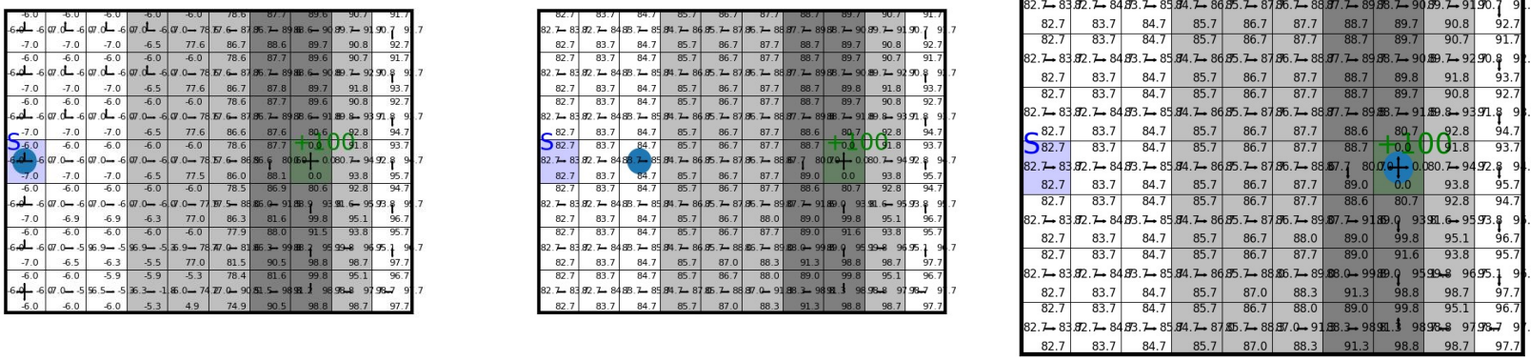
2.2 EXPERIMENTATION

The environment in which the agent learns the policy is StochasticWindyGridworld. We perform the experiment with the hyperparameter values of γ set to 1.0 and $threshold$ set to 0.001. Then we let the algorithm run till convergance. In the above figures we can see how the agent interacts with the environment and tries to reach the goal state.

The iteration begins with the agent present at the start state as shown in Figure1a and as it interacts with the environment it tries to learn the optimum policy while updating the value function and the Q-table.

In the Figure1b the agent is interacting with the environment and the value of each state-action pair is getting evaluated using Bellman equation, the algorithm iterates until the change in Q-values falls below the specified threshold.

Finally in the Figure1c we can see that the agent has reached the goal state and now the iteration terminates with the agent having learned the optimum policy.



(a) Q value iteration at start state

(b) Q value iteration in progress

(c) Q value iteration reaching terminal state

Figure 1: Q value iteration (Dynamic programming)

2.3 Results and Discussion

- The converged optimal value at the start i.e $V^*(s = 3)$ obtained in our case is **83.6782**. To compute this value, we check each action's Q-value in state 3 and select the action that maximizes the Q-value.
- The value obtained for the mean reward per timestep under the optimal policy is **4.3157**. It is computed as $totalreward/timestep$. Here total reward is the cumulative sum of all the rewards obtained till the agent reaches the goal state, and timestep is the number of steps it took the agent to reach the goal state.
- Before the start of the iteration we define a variable $done$ initialized with value $False$. With every step that the agent takes, this variable stores the return value which is boolean of whether the agent has reached the goal state or not. Once this value becomes $True$, we stop the iteration which indicates that the terminal state is reached.
  1. Another way to check if we have reached the terminal state is checking the reward values. We can provide a certain reward value which indicates that the agent has reached the goal.
- If we change the terminal location from [7,3] to [6,2], we observe a significant increase in the time it takes the agent to reach the goal state, also the mean reward decreases. This could be because there are different challenges in the new goal state.

3 Exploration

Q-Learning is a form of Off-policy learning algorithm that does not uses the Makrov's transitiion probability distribution, it is more complicated; it may learn its policy from actions that are different from the one just taken. The algorithm balances exploration and exploitation using an exploration-exploitation strategy but it evaluates states as if a greedy policy is used always, even when the actual behavior performed an exploration step. The exploration-exploitation strategy is used to balance exploration of new states and exploitation of the current knowledge. The Q-values are updated using the Q-value of the next state s_{t+1}, and the $greedy\ action$.
The Q-learning update formula is:-

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

where,

- Q(s, a) is the Q-value for state s and action a.
- α is the learning rate which controls the steps size of the Q-value update. If set to 0, agent will not learn from the next action, if set to 1, agent only considers the recent action.
- $Q(s_{t+1}, a_{t+1})$ is the Q-value for the next state s_{t+1} and next action a_{t+1}.

3.1 Procedure

3.1.1 Action Selection

The procedure in our task starts from action selection, we use two types of policies ϵ-greedy and Boltzmann. Both of these are defined as:-

- ϵ-greedy

$$\pi(a|s) = \begin{cases} 1.0 - \frac{\epsilon \cdot |A|^{-1}}{|A|}, & \text{if } a = \arg\max_{b \in A} Q(s, b) \\ \frac{\epsilon}{|A|}, & \text{otherwise} \end{cases}$$

  where, ϵ = 0 gives a greedy policy and ϵ = 1 gives gives a uniform/random policy.. Here, to choose between explore and exploit we sample a random value, if that value is less than ϵ we explore otherwise we select action with greedy policy.
- Boltzmann Policy:

$$\pi(a|s) = \frac{e^{Q\wedge(s,a)/\tau}}{\sum_{b \in A} e^{Q\wedge(s,b)/\tau}}$$

where, for τ = 0 the policy becomes greedy and for $\tau = \infty$ the policy becomes uniform/random.

3.1.2 UPDATE

After executing an action, the environment gives you new data, in the form of the observed reward and next state. We use this data and compute the new target and then perform the **tabular learning update**

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

3.1.3 ALGORITHM:

Now we define the steps in the Tabular Q-learning algorithm.

Algorithm 2: Tabular Q-learning

Input: Exploration parameter, learning rate $\alpha \in (0, 1]$, discount parameter $\gamma \in [0, 1]$, total budget.
$Q^{\wedge}(s, a) \leftarrow 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
while $budget$ **do**

1. $a \sim \pi(a|s)$
2. $r, s' \sim p(r, s'|s, a)$
3. $Q^{\wedge}(s, a) \leftarrow Q^{\wedge}(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q^{\wedge}(s', a') - Q^{\wedge}(s, a)]$
4. **if** s' $is\ terminal$ **then**
5. $\quad s \sim p_0(s)$
6. **else**
7. $\quad s \leftarrow s'$
8. **end**

end
Return $Q^{\wedge}(s, a)$

The algorithm is explained as follows:-

- Initialize the Q-value table, Q^(s, a), for all states s and actions a to zero.
- call the state values, action values and rewards function for the task.
- Start the loop and do till convergance:
  1. Sample an action a from the policy to take in the current state. This could be done using an ϵ-greedy or softmax.
  2. Once you receive the next reward and state, use them to move ahead in the environment.
  3. Perform the Q-value update for the current state-action pair using the Q-learning update rule and maximize the Q-value function.
  4. Check if the next state is terminal, if yes then rest the environment. If not then set the current state to the next state.
- Once convergence is achieved, we can obtain the optimal policy by selecting the action with the highest Q-value for each state.

3.2 EXPERIMENT

We start our experiment first with checking the performance of the Q-learning algorithm for ϵ-greedy policy for epsilon values in [0.03,0.1,0.3] and the hyperparameter values of γ set to 1.0, learning rate(α) set to 0.1. We define an evaluation interval of 1000 after which we evaluate the performance of our agent using the **greedy policy** for 30 episodes and calculate the mean and see if our agent is learning properly.

We then perform the experiment for softmax policy for the temprature values in [0.01,0.1,1.0], the value for the remaining hyperparameter stay same for both the policies. Also we similarly evaluate the performance of the agent for every evaluation interval and obtain the mean.
We run the algorithm for 20 repetitions and with a budget of 70000 and take the mean of the returns of the repetitions for plotting.

3.3 RESULTS AND DISCUSSION

After the experiments, we obtained the results which are depicted in Figure2. The first trend that we observe is that the mean return is negative at the start, but as the steps reach 10000, it increases exponentially. From the figure we can make the following observations:-

- In case of ϵ-greedy the algorithm converged faster for the value which was 0.1 i.e. when the exploration was less and exploitation was higher, so agent chose best actions for the states. However, after convergence, we can see from the graph that there is not much difference in the performance.
- Looking at the plotting for softmax, we can see that for τ = 0.01 and 0.1, the algorithm converged fastest. However, for τ = 1.0 the algorithm took longer to converge.
- From the graph we can see that overall ϵ-greedy converged faster than softmax for all the values for both policies, hence it seems to be the better choice.
- On comparing with Dynamic Programming, one can say that the performance of RL is average at best, since in RL the policy is learned from interaction with the environment, which can often lead to badly optimized policies if the agent fails to explore all the possible actions and states in the environment.
- After adding multiple goals, we observed the following scenarios:-
  1. The second goal state at location (3,2), with a reward of +5 which was a lot closer to the starting location and also did not have much influence of wind.

2. Furthermore, it was observed that once the agent reached the second goal state then even after resetting the environment, the agent stopped exploring completely and only tried to reach the second goal state using different paths until the number of timesteps finished. It seemed as if the agent was not using the exploration parameter at all.

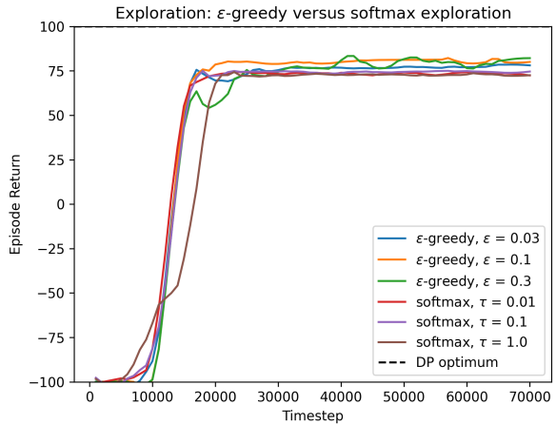


Figure 2: Q-learning Exploration

4 BACK-UP: ON-POLICY VERSUS OFF-POLICY TARGET

In this section we will look at the on-policy learning algorithm **SARSA** that stands for stands for state-action-reward-state-action. However, like Q-learning, it is a model-free reinforcement learning algorithm. On-policy learning updates the policy with the action values of the policy. The SARSA update formula is:-

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

In on policy learning, the behaviour policy guides the selection of the action, evaluates it in the environment, and follows the actions. The behaviour policy is not specified by any particular formula, but ϵ-greedy or any other policy that balances exploration-exploitation is used. In SARSA, the Q-values are updated using the Q-value of the next state s_{t+1} and the current policy's action. The primary advantage of on-policy learning is its predictive behavior.

4.1 PROCEDURE

4.1.1 UPDATE

In SARSA, the bootstrapping at next state happens as: SARSA backs up the value of the action we actually take (which may be exploratory and not the one with the currently optimal estimate). SARSA therefore learns the value function of the policy we actually execute (which includes exploration). The back-up equation for SARSA, given observations $\langle s_t, a_t, r_t, s_{t+1}, a_{t+1} \rangle$ is:-

$$G_t = r_t + \gamma \cdot Q\hat{}(s_{t+1}, a_{t+1})$$

where, G_t is the new back-up estimate/target.
Now we substitute the value of G_t and do the Q-value update using the SARSA update formula 4

4.1.2 ALGORITHM

Now we will discuss the algorithm for SARSA ON-policy update. The algorithm is defined as:-

The algorithm is explained as follows:-

- Initialize the Q-value table, Q^(s, a), for all states s and actions a to zero
- call the state values, action values and rewards function for the task.
- sample an initial state from the environment and then sample action for the particular state based on a policy such as ϵ-greedy or softmax.
- Start the loop and do till convergance:
  1. Using the action sampled before the loop, simulate the environment and observe the reward and the next state.
  2. using the next state, sample the next action using the exploration-exploitation strategy.
  3. Perform the Q-value update for the current state-action pair using the SARSA update rule 4.
  4. Check if the next state is terminal, if yes then rest the environment and sample initial action. If not then set the current state to the next state and current action to the next action.
- Once convergence is achieved, we can obtain the optimal policy by selecting the action with the highest Q-value for each state.

4.2 EXPERIMENT

For SARSA, instead of just observing the performance of the algorithm for different policies, we perform a comparison with Q-learning.
We only use the ϵ-greedy policy with value set to 0.1 and observe the performance of both algorithms for different learning rates of [0.03,0.1,0.3]. We set the hyperparameter value of γ to 1.0 and set an evaluation interval of 1000 after which we evaluate the performance of our agent using the **greedy** policy for 30 episodes and calculate the mean and see if our agent is learning properly. We run the algorithm for 20 repetitions with budget of 70000 and take the mean of the returns of the repetitions for plotting.

Algorithm 3: Tabular SARSA.

Input: Exploration parameter, learning rate $\alpha \in (0, 1]$, discount parameter $\gamma \in [0, 1]$, total budget.
$Q^{\wedge}(s, a) \leftarrow 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
$a \sim \pi(a|s)$
$s \sim p_0(s)$
while *budget* **do**

1. $r, s' \sim p(r, s'|s, a)$
2. $a' \sim \pi(a'|s')$
3. $Q^{\wedge}(s, a) \leftarrow Q^{\wedge}(s, a) + \alpha \cdot [r + \gamma \cdot Q^{\wedge}(s', a') - Q^{\wedge}(s, a)]$
4. **if** s' *is terminal* **then**
5. $\quad s \sim p_0(s)$
6. $\quad a \sim \pi(a|s)$
7. **else**
8. $\quad s \leftarrow s'$
9. $\quad a \leftarrow a'$
10. **end**

end
Return $Q^{\wedge}(s, a)$

4.3 Results and Discussion

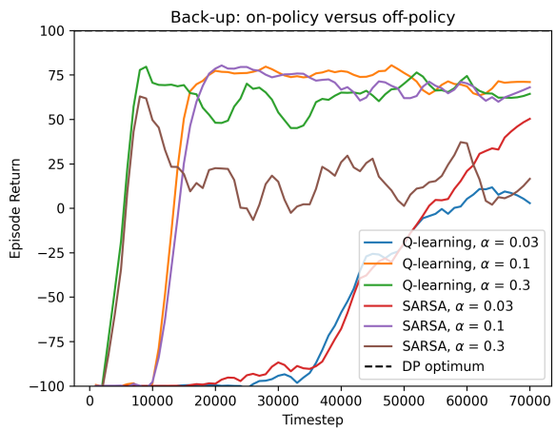


Figure 3: On Off policy comparison

Following the experiments, the obtained results are depicted in 3. The first thing we need to know is that Sarsa gets higher average rewards per run than Q-learning, hence Q-learning may perform in a suboptimal way, whereas the Sarsa's performance is more optimal. From the figure we can make the following observations:-

- For the learning rate 0.03, both Q-learning and SARSA take the longest to converge and the episode return does not go beyond 50.
- For the learning rate 0.1, both algorithms gave similar performance with faster convergence and good episode return.
- For the learning rate 0.3, we can observe that the performance of Q-learning was better than SARSA.

From the above observations we can say that overall Q-learning gives a better performance, hence it should be the preferred choice when compared with SARSA.

5 Back-up: Depth of target

5.1 n-step Q-learning

Till now we have seen temporal difference bootstrapped Q-functions that refine the values of previous steps with rewards after each step. However with this approach, the old reward value still resides which can lead to biasing, thus TD is a high-bias/low variance method. Further we will see the Monte Carlo algorithm which is a low-bias/high-variance algorithm, hence its action choices are unbiased. But we need something which is in the middle ground. This is where the $n - step$ approach is useful, it does not sample a full episode, and also not a single step, but sample a few steps at a time before using the reward values.

Algorithm 4: Tabular n-step Q-learning

Input: Exploration parameter $\epsilon \in (0, 1]$, learning rate $\alpha \in (0, 1]$, discount parameter $\gamma \in [0, 1]$, maximum episode length T, target depth n.
$Q^\wedge(s, a) \leftarrow 0, \forall s \in S, a \in A$
while budget **do**
    $s_0 \sim p_0(s)$
    for $t = 0$ to $T - 1$ **do**
        $a_t \sim \pi(a|s_t)$
        $r_t, s_{t+1} \sim p(r, s'|s_t, a_t)$
        if s_{t+1} is terminal **then**
            break
        end if
    end for
    $T_{ep} \leftarrow t + 1$
    for $t = 0$ to $T_{ep} - 1$ **do**
        $m = \min(n, T_{ep} - t)$
        if s_{t+m} is terminal **then**
            $G_t \leftarrow \sum_{i=0}^{m-1} \gamma^i \cdot r_{t+i}$.
        else
            $G_t \leftarrow \sum_{i=0}^{m-1} \gamma^i \cdot r_{t+i} + \gamma^m \cdot \max_a Q^\wedge(s_{t+m}, a)$
        end if
        $Q^\wedge(s_t, a_t) \leftarrow Q^\wedge(s_t, a_t) + \alpha \cdot [G_t - Q^\wedge(s_t, a_t)]$
    end for
end while
Return: $Q^\wedge(s, a)$ =0

Algorithm 5: Tabular Monte Carlo reinforcement learning

Input: Exploration parameter $\epsilon \in (0, 1]$, learning rate $\alpha \in (0, 1]$, discount parameter $\gamma \in [0, 1]$, maximum episode length T.
$Q^\wedge(s, a) \leftarrow 0, \forall s \in S, a \in A$
while budget **do**
    $s_0 \sim p_0(s)$
    for $t = 0$ to $T - 1$ **do**
        $a_t \sim \pi(a|s_t)$
        $r_t, s_{t+1} \sim p(r, s'|s_t, a_t)$
        if s_{t+1} is terminal **then**
            break
        end if
    end for
    $G_{t+1} \leftarrow 0$
    for $i = t$ to 0 **do**
        $G_i \leftarrow r_i + \gamma \cdot G_{i+1}$
        $Q^\wedge(s_i, a_i) \leftarrow Q^\wedge(s_i, a_i) + \alpha \cdot [G_i - Q^\wedge(s_i, a_i)]$
    end for
end while
Return: $Q^\wedge(s, a)$

5.1.1 PROCEDURE

n-step methods can vary depending on the way you use bootstrap, we are using the Q-learning, which computes the following target:

$$G_t = \sum_{i=0}^{n-1} \gamma^i \cdot (r_{t+i} + \gamma^n \max_a Q(s_{t+n}, a))$$

and then the standard tabular update.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[G_t - Q(s_t, a_t)\right]$$

5.2 MONTE-CARLO

Monte Carlo is a reinforcement learning algorithm that does not use bootstrapping it performs a full episode with random action choices before it uses the reward. Since it has no knowledge about the environment's dynamics, its action choices are unbiased. Thus the $Monte\ Carlo$ update is as follows:-
First we perform the target computation

$$G_t = \sum_{i=0}^{\infty} \gamma^i \cdot r_{t+i}$$

Then using the standard tabular update 5.1.1 equation, we update the Q-values.

5.3 ALGORITHM

5.3.1 N-STEP

The n-step algorithm is be explained as follows:-

- Initialize the Q-value table, Qˆ(s, a), for all states s and actions a to zero.
- call the state values, action values and rewards function for the task.
- Start the loop and do till convergance:
    1. sample an initial state.
    2. Define lists for action,states and rewards.
    3. sample action for the current state using ϵ-greedy or softmax policy.
    4. Observe the reward and the next state and append the values to the respective lists.
    5. if the state is terminal, then break and call the update function
        (a) compute n-step returns for the last states for target
    6. using the n-step return, update the Q-values for each state action pair.
- Once convergence is achieved, we can obtain the optimal policy by selecting the action with the highest Q-value for each state.

5.3.2 MONTE CARLO

The Monte Carlo algorithm is be explained as follows:-

- Initialize the Q-value table, Qˆ(s, a), for all states s and actions a to zero.
- call the state values, action values and rewards function for the task.
- Start the loop and do till convergence:
  1. sample an initial state.
  2. Define lists for action,states and rewards.
  3. sample action for the current state using ϵ-greedy or softmax policy.
  4. Observe the reward and the next state and append the values to the respective lists.
  5. if the state is terminal, then break and call the update function.
     (a) Compute the target G_t as the sum of rewards obtained from time $t + 1$ till the end of the episode.
  6. Using the return obtained from the Monte Carlo update, update the Q-values for each state-action pair.
- Once convergence is achieved, we can obtain the optimal policy by selecting the action with the highest Q-value for each state.

5.4 EXPERIMENT

For this part we again do a comparison of the performance of the methods which are n-step method and Monte Carlo. We run the n-step algorithm for three different step sizes which are [1,3,10]. The policy that we use for both methods is ϵ-greedy with value set to 0.05. The remaining hyperparametes that we use are the learning rate set to 0.1, max episode length set to 100 and γ set to 1.0
We run the algorithm for 20 repetitions and with a budget of 70000 and take the mean of the returns of the repetitions for plotting.

5.5 RESULTS AND DISCUSSION

After performing the experiments, the obtained results have been depicted in Figure4 and we can make the following observations:-

- From the figure it is clearly visible that n-step method performs extremely well when compared to Monte Carlo, the line is completely flat for Monte Carlo, and this is expected since it requires the entire episode to complete before the agent can update the estimates of the Q-value function.
- When comparing the performance of of different step values for n-step, we can observe that:-
  1. For step value = 10, the performance of the algorithm doe seem to closer to step values 1 and 3, however we can see at the end that the graph for it begins to dip.
  2. This behaviour can happen because since step values of 1 and 3 are very small, and the algorithm does not need to wait for a long time to perform the updates, hence the graphs converge faster. So for 10 step the graph is not very smooth and seems to be dipping with the increasing step size.
- On comparing our results with Q-learning implementation and n-step Q-learning implementation with n = 1 we do observe the difference, however the reason for this could be the ϵ value that was used in both methods. The ϵ values used for Q-learning were 0.03,0.1,0.3 and for n-step Q-learning was 0.05 so as we defined earlier that for Q-learning the algorithm performed better for bigger values of ϵ, hence for a lower value in n-step learning there was slight difference in the performance.

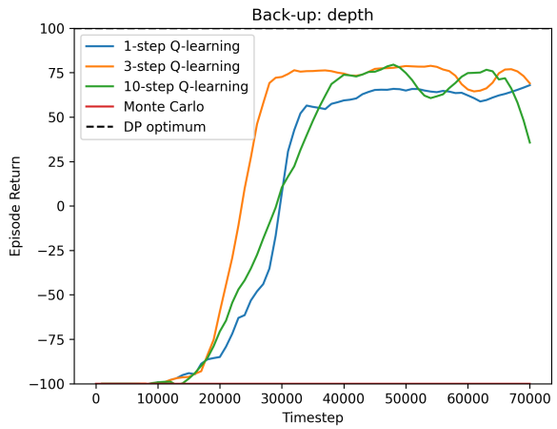


Figure 4: Depth

6 REFLECTION

6.1 DP VERSUS RL

Both the methods of Dynamic Programming and Reinforcement Learning are important approaches used for solving sequential decision-making problems and both are used to solve many real world problems.

- The major advantage the DP has over RL is that since DP is a model based method, it has knowledge of both the transition matrix and the rewards, hence it can perform much better as it can obtain the optimal solutions offline and do the learning without a lot of trial and error. Whereas since RL is a model free method, it has to learn by interacting with the environment and if too much exploration is used, then convergence might not be possible.

- The disadvantage however is that DP assumes that the knowledge of the transition matrix and the rewards is always available, which to honest is not always achievable in real life. Also problems that are extremely large that have millions of states are not suitable for DP.

6.2 EXPLORATION

- ϵ-greedy policy works by choosing between selecting a greedy action with high rewards which is the exploitation, and a random action with probability ϵ which is exploration. The benefit of this is that we have a guarantee that all actions with non-zero probability will be sampled. But the catch here is that all the parameters must be tuned and set correctly, otherwise we may get sub-optimal results.
- With Softmax, we have a more exploration heavy method in which the exploration method gives a probability to an action on the basis of their rewards. The main advantage with Softmax is that actions with high values are pushed for exploration. But this also means that the method can be computationally expensive then ϵ-greedy as we need to compute and normalize probability distribution over all actions.
- Personally i would prefer ϵ-greedy method since as per my experiments I observed that it performed well for all the different ϵ values.
- Other methods that we can try our Thompson sampling: The agent keeps track of a belief over the probability of optimal actions and samples from this distribution or Entropy loss term in which we add an entropy term into the loss function, encouraging the policy to take diverse actions.

6.3 EXPLORATION 2:

In case of Dynamic Programming, when we change the value of default reward per step to 0 we observe that all the values in the gride become 100, and the convergence happens quickly. This could happen because now the agent is not getting penalized rewards per step and the overall value becomes fairly positive enabling the agent to get better best actions for each state and it can do exploration without compromising on the rewards.

6.4 DISCOUNT FACTOR

After making the changes and performing the experiment for Dynamic Programming as per john's requirement, we observed that the agent reached the goal state for both the values of γ, however for the value of 1, the agent found the shortest path slightly faster.

6.5 BACK-UP, ON-POLICY VERSUS OFF-POLICY

- In on policy methods like SARSA, the aim is to improve the decision making policy, the value function is updated based on the actions that the agent actually takes. In on-policy, the learning happens for the current policy by following that policy. Since the method follows the policy, the convergence to optimal policy has high possibility. Even if the environment is stochastic or unstable.
- In off-policy learning such as Q-learning, the Q-value function is updated on the basis of maximum future reward. The policy remains more flexible even if different paths appear. In off-policy, the learning of the optimal policy happens while following a different policy. Due to all these factors, the off-policy methods are more efficient for scenarios such as robotics, where off-policy is more useful in making the next movement.
- The disadvantage of on-policy method is that it can get trapped in local minima, and since it follows the current policy, the update is slower.
- In case of off-policy, the main disadvantage is that if the agent becomes too explorative, then convergence becomes difficult.
- The main difference between on-policy and off-policy method lies in the way in which they both update the Q-value function. In case of on-policy, the Q-value function is updated based on the actions that the agent actually takes. Whereas, for off-policy, the Q-value function is updated on the basis of maximum future reward
- n-step learning is an off-policy algorithm, this is because it is a form of Q-learning method and the learning of the optimal policy happens while following a different policy. However, as stated earlier that n-step lies in the middle ground as it does not sample a full episode, and also not a single step, but sample a few steps at a time before using the reward values. Hence, it can use the benefits of both types of policies while avoiding the disadvantages.

6.6 BACK-UP, TARGET DEPTH

All three methods of One-step, n-step, and Monte Carlo are used in RL to calculate the value function.

- In one-step methods, the transition function defines a one-step transition, as each action (from a certain old state) deterministically leads to a single new state. Due to this, one-step methods have the ability to converge quickly to optimal policy. However since the method is heavily reliant on the immediate rewards, it is biased.
- The n-step methods are best in balancing the bias-variance trade-off. Since they n-step do not sample a full episode, and also not a single step, but sample a few steps at a time before using the reward values. The n-step algorithm has medium bias and medium variance. However, it is important to choose the value of step size n correctly.
- The Monte Carlo method does not use bootstrapping. It performs a full episode with many random action choices before it uses the reward. This gives the Monte Carlo method the advantage that it's completely un-biased. However, since they perform a full episode, the variance is high and convergence takes extremely long.
- The bias-variance trade-off is explained as: a model that underfits the data has high bias, whereas a model that overfits the data has high variance. The one-step methods have high bias and low variance. The Monte Carlo method has high variance and low bias and the n-step method is in the middle with medium bias and medium variance.
- For our task the method i would prefer is n-step since it converges to optimal policy at a good rate and also gives better returns per episode. The method that converges information faster is dependent on the conditions that are set, for our tasks the n-step does converge faster.

6.7 CURSE OF DIMENSIONALITY

The tabular RL algorithms that we studied almost all have good convergance. Also they provide proper representation of the Q-value function. However they run into trouble when the effect of an exponential need for observation data causes the dimensionality grows. This has been called the curse of dimensionality. This means that the number of unique points) of a space scales exponentially in the dimensionality of the problem. However, the number of observations typically does not grow at such scale. In case of a deep reinforcement learning, the network can be trained to approximate the value function or policy using high-dimensional input states. This can enable the agents to learn how to achieve their objectives

(Sutton & Barto, 2018). (Plaat, 2022) (Moerland, 2021) Weng (2020)

REFERENCES

Thomas Moerland. *Continuous Markov Decision Process and Policy Search*. 2021.

Aske Plaat. *Deep Reinforcement Learning*. 2022.

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. 2018.

Lilian Weng. Exploration strategies in deep reinforcement learning. *lilianweng.github.io*, Jun 2020. URL https://lilianweng.github.io/posts/2020-06-07-exploration-drl/.